Faculty of Engineering and Applied Science

Department of Electrical, Computer and Software Engineering

SOFE 3950U – Operating Systems

Tutorial Assignment 9

Due Date: April 2nd, 2025

Course Instructor: Dr Khalid Hafeez

CRN: 74171- Group # 6

Student Name	Student Number
Richard Edom	100927810
Geraline Chavez	100890130
Nicholas Furtado	100908880
Eshaan Srirangam	100863051

Conceptual Questions

1. ***Explain what OpenMP is, what are its benefits?***

OpenMP is an application programming interface, which is used for parallel programming in C, C++, and Fortran programming languages. By using the OpenMP interface, it is possible to execute multiple threads in a program at the same time on a shared memory system. In this way, the performance of the program can be improved by executing multiple tasks on multiple CPU cores rather than sequentially.

Benefits of OpenMP

- Improves performance by executing parts of a program in parallel.
- Makes efficient use of multiple CPU cores.
- Easier to use since it is based mainly on compiler directives like #pragma.
- Easier to add to existing code since it requires minimal modifications.
- Suitable for shared memory systems, which are found in most computers.

2. ***What are the #pragma definitions, what do they do?***

#pragma directives are instructions in C languages for example, that tell the compiler specific ways to handle certain tasks. Unlike a regular command, they are compiler specific and therefore not all compilers support the same #pragma options. They are typically used for optimization levels, warnings, memory alignments. Etc.

3. ***Write the OpenMP #pragma definition to execute a loop in parallel.***

   The #pragma omp parallel for directive tells OpenMP to split the loop iterations across multiple threads so they run at the same time instead of sequentially.

4. ***What does the reduction do in the #pragma definition in OpenMP?***

   The reduction clause in OpenMP creates a private copy of the specified variable for each thread, allowing each thread to safely accumulate its own partial result without interfering with other threads. Once all threads have finished their work, OpenMP automatically combines all the private copies into a single final result using the specified operator. Without reduction, multiple threads writing to the same shared variable would cause a race condition, leading to incorrect and unpredictable results.

5. ***Explain the critical and private() declarations used in OpenMP -Richard***

   The critical directive defines a block of code that can be executed only by one thread at a time. When a thread enters a critical section, all other threads must wait until it finishes

before they can enter, which prevents race conditions on shared resources like output statements but incurs a performance cost due to forced serialization. The Private() clause, on the other hand, gives each thread its own independent, uninitialized copy of the specified variable. This ensures that when one thread modifies the variable, it does not overwrite or corrupt the value being used by another thread, effectively eliminating a race condition on that variable.

Application Questions

1.

demo_1.c demo_2.c

C: > Users > reyes > Desktop > OS_TUT > TUT_9 > demo_1.c

```c
#include <stdio.h>
#include <omp.h>

int main (int argc, char *argv[])
{
    printf("This statement will always print.\n");

    // Checks if OpenMP is enabled when compiling the program
    #ifdef _OPENMP
    printf("This statement will only print when compiled with \"-fopenmp\"\n");
    #endif

    return 0;
}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS SONARQUBE

```
-bash: cd: C:UsersreyesDesktopOS_TUTTUT_9: No such file or directory
reyes@Geraline:/mnt/c/Users/reyes$ cd C:\Users\reyes\Desktop\OS_TUT\TUT_9
-bash: cd: C:UsersreyesDesktopOS_TUTTUT_9: No such file or directory
reyes@Geraline:/mnt/c/Users/reyes$ cd reyes\DesktopOS_TUT\TUT_9
-bash: cd: reyesDesktopOS_TUTTUT_9: No such file or directory
reyes@Geraline:/mnt/c/Users/reyes$ cd /mnt/c/Users/reyes/Desktop/OS_TUT/TUT_9
reyes@Geraline:/mnt/c/Users/reyes/Desktop/OS_TUT/TUT_9$ gcc -Wall -Wextra -std=c99 -fopenmp demo_1.c -o demo1 -lm
demo_1.c: In function ‘main’:
demo_1.c:4:15: warning: unused parameter ‘argc’ [-Wunused-parameter]
    4 | int main (int argc, char *argv[])
      |           ~~~~^~~~
demo_1.c:4:27: warning: unused parameter ‘argv’ [-Wunused-parameter]
    4 | int main (int argc, char *argv[])
      |                     ~~~~~~~^~~~~~
reyes@Geraline:/mnt/c/Users/reyes/Desktop/OS_TUT/TUT_9$ ./demo1
This statement will always print.
This statement will only print when compiled with "-fopenmp"
reyes@Geraline:/mnt/c/Users/reyes/Desktop/OS_TUT/TUT_9$
```

demo_1.c | demo_2.c

C: > Users > reyes > Desktop > OS_TUT > TUT_9 > demo_2.c

```c
#include <stdio.h>
#include <omp.h>

int main (int argc, char *argv[])
{
    int num = 0; // Watch out for race conditions!
    int thread_num, nthreads = 16;

    // If OpenMP is enabled, set how many threads will be used
    #ifdef _OPENMP
    omp_set_num_threads(nthreads);
    #endif

    printf("Testing OpenMP, you should see each thread print...\n");

    // Starts the parallel part of the program
    #pragma omp parallel
    {
        // only one thread runs this part at a time
        #pragma omp critical
        {
            // Only runs this part if OpenMP is active
            #ifdef _OPENMP
            thread_num = omp_get_thread_num();
            num += 1;
            printf("This thread = %d, num = %d\n", thread_num, num);
            #endif
        }
    }

    return 0;
}
```

```
reyes@Geraline:/mnt/c/Users/reyes/Desktop/OS_TUT/TUT_9$ gcc -Wall -Wextra -std=c99 -fopenmp demo_2.c -o demo2 -lm
demo_2.c: In function ‘main’:
demo_2.c:4:15: warning: unused parameter ‘argc’ [-Wunused-parameter]
    4 | int main (int argc, char *argv[])
      |           ~~~~^~~~
demo_2.c:4:27: warning: unused parameter ‘argv’ [-Wunused-parameter]
    4 | int main (int argc, char *argv[])
      |                     ~~~~~~~~^~~~~~
reyes@Geraline:/mnt/c/Users/reyes/Desktop/OS_TUT/TUT_9$ ./demo2
Testing OpenMP, you should see each thread print...
This thread = 13, num = 1
This thread = 12, num = 2
This thread = 9, num = 3
This thread = 2, num = 4
This thread = 10, num = 5
This thread = 14, num = 6
This thread = 0, num = 7
This thread = 3, num = 8
This thread = 15, num = 9
This thread = 6, num = 10
This thread = 4, num = 11
This thread = 8, num = 12
This thread = 1, num = 13
This thread = 7, num = 14
This thread = 5, num = 15
This thread = 11, num = 16
reyes@Geraline:/mnt/c/Users/reyes/Desktop/OS_TUT/TUT_9$
```

2.

```
nicho@NicksLenovo:/mnt/c/Users/nicho/Onedrive/Documents/Github/OS_TUT9/OS_TUT_9_CRN-74171$ gcc -fopenmp OS_TUT9_Q2.c -lm -o tut9
nicho@NicksLenovo:/mnt/c/Users/nicho/Onedrive/Documents/Github/OS_TUT9/OS_TUT_9_CRN-74171$ ./tut9 1
./tut9 2
./tut9 4
./tut9 8
Execution time with 1 threads: 2.837449 seconds
y[0] = 0.000000, y[n-1] = 4.098895
Execution time with 2 threads: 1.283040 seconds
y[0] = 0.000000, y[n-1] = 4.098895
Execution time with 4 threads: 0.642202 seconds
y[0] = 0.000000, y[n-1] = 4.098895
Execution time with 8 threads: 0.387987 seconds
y[0] = 0.000000, y[n-1] = 4.098895
nicho@NicksLenovo:/mnt/c/Users/nicho/Onedrive/Documents/Github/OS_TUT9/OS_TUT_9_CRN-74171$
```

OS_TUT9_Q2.c U

C: > Users > nicho > OneDrive > Documents > GitHub > OS_TUT9 > OS_TUT_9_CRN-74171 > OS_TUT9_Q2.c > main(int, char * [])

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

int main(int argc, char *argv[])
{
    if (argc < 2) {
        printf("Usage: %s <num_threads>\n", argv[0]);
        return 1;
    }

    int nthreads = atoi(argv[1]);

    #ifdef _OPENMP
    omp_set_num_threads(nthreads);
    #endif

    long int n = 100000000;

    double *y = (double*) malloc(n * sizeof(double));
    if (y == NULL) {
        printf("Memory allocation failed\n");
        return 1;
    }

    double dx = 1.0 / (n + 1);

    double start = omp_get_wtime();

    double x;
    #pragma omp parallel for private(x)
    for (long int i = 0; i < n; i++) {
        x = i * dx;

        y[i] = exp(x) * cos(x) * sin(x) * sqrt(5.0 * x + 6.0);
    }

    double end = omp_get_wtime();

    printf("Execution time with %d threads: %f seconds\n", nthreads, end - start);

    printf("y[0] = %f, y[n-1] = %f\n", y[0], y[n-1]);

    free(y);

    return 0;
}
```

3.

```c
#include <stdlib.h>
#include <stdio.h>
#include <math.h>
#include <omp.h>

#define N 100000000

int main(void)
{
    int *x = (int *)malloc(N * sizeof(int));
    if (x == NULL) {
        printf("Memory allocation failed!\n");
        return 1;
    }

    srand(42);
    for (int i = 0; i < N; i++) {
        x[i] = rand() % 101;
    }

    double norm_serial = 0.0;
    double start_serial = omp_get_wtime();
    for (int i = 0; i < N; i++) {
        norm_serial += abs(x[i]);
    }
    double end_serial = omp_get_wtime();
    double time_serial = end_serial - start_serial;
```

```
    printf("Serial norm: %f\n", norm_serial);
    printf("Serial time: %f seconds\n\n", time_serial);

    double norm_parallel = 0.0;
    omp_set_num_threads(16);

    double start_parallel = omp_get_wtime();
    #pragma omp parallel for reduction(+:norm_parallel)
    for (int i = 0; i < N; i++) {
        norm_parallel += abs(x[i]);
    }
    double end_parallel = omp_get_wtime();
    double time_parallel = end_parallel - start_parallel;

    printf("Parallel norm: %f\n", norm_parallel);
    printf("Parallel time: %f seconds\n\n", time_parallel);

    printf("Time difference: %f seconds\n", time_serial - time>
    printf("Speedup: %.2fx\n", time_serial / time_parallel);

    free(x);
    return 0;
}
```

```
richardedom@ubuntu:~/tutorial9$ nano question3.c
richardedom@ubuntu:~/tutorial9$ gcc -Wall -Wextra -std=c99 -fop
enmp question3.c -o question3 -lm
./question3
Serial norm: 5000184656.000000
Serial time: 0.354891 seconds

Parallel norm: 5000184656.000000
Parallel time: 0.102346 seconds

Time difference: 0.252545 seconds
Speedup: 3.47x
richardedom@ubuntu:~/tutorial9$
```

4.

C: > Users > eshaa_ntl31s1 > OneDrive > Documents > T9q4.c > main(int, char * [])

```c
#include <stdio.h>
#include <omp.h>

#define N 100

int main (int argc, char *argv[])
{
    int a[N][N], b[N][N];
    int c_parallel[N][N], c_sequential[N][N];

    int nthreads = 8;
    #ifdef _OPENMP
    omp_set_num_threads(nthreads);
    #endif

    // Initialize matrices
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            a[i][j] = i + j;      // based on index
            b[i][j] = i - j;      // based on index
            c_parallel[i][j] = 0;
            c_sequential[i][j] = 0;
        }
    }

    printf("Running parallel matrix multiplication...\n");

    // ✅ Parallel multiplication (OpenMP)
    #pragma omp parallel for collapse(2)
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            for(int k = 0; k < N; k++) {
                c_parallel[i][j] += a[i][k] * b[k][j];
            }
        }
    }
```

```c

    printf("Running sequential matrix multiplication...\n");

    // ✅ Sequential version (verification tool)
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            for(int k = 0; k < N; k++) {
                c_sequential[i][j] += a[i][k] * b[k][j];
            }
        }
    }

    // ✅ Verify correctness
    int correct = 1;
    for(int i = 0; i < N; i++) {
        for(int j = 0; j < N; j++) {
            if(c_parallel[i][j] != c_sequential[i][j]) {
                correct = 0;
                break;
            }
        }
    }

    if(correct)
        printf("Verification PASSED: Parallel result is correct.\n");
    else
        printf("Verification FAILED: Results do not match.\n");

    // Print sample value
    printf("Sample result c[0][0] = %d\n", c_parallel[0][0]);

    return 0;
}
```

```
eshaa_ntl31s1@LAPTOP-DM53QFHD:/mnt/c/Users/eshaa_ntl31s1/OneDrive/Documents$ gcc -fopenmp T9q4.c -o T9q4
eshaa_ntl31s1@LAPTOP-DM53QFHD:/mnt/c/Users/eshaa_ntl31s1/OneDrive/Documents$ ./T9q4
Running parallel matrix multiplication...
Running sequential matrix multiplication...
Verification PASSED: Parallel result is correct.
Running parallel matrix multiplication...
Running sequential matrix multiplication...
Verification PASSED: Parallel result is correct.
Running sequential matrix multiplication...
Verification PASSED: Parallel result is correct.
Verification PASSED: Parallel result is correct.
Sample result c[0][0] = 328350
```

5.

```c
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <omp.h>

int main(int argc, char *argv[])
{
    // ✅ Check command line argument
    if(argc < 2) {
        printf("Usage: %s <num_threads>\n", argv[0]);
        return 1;
    }

    int num_threads = atoi(argv[1]);
    omp_set_num_threads(num_threads);

    long n = 100000000; // 100 million
    double dx = 1.0 / (n + 1);

    FILE *fp = fopen("calculations.txt", "w");
    if(fp == NULL) {
        printf("Error opening file\n");
        return 1;
    }

    printf("Running with %d threads...\n", num_threads);

    #pragma omp parallel
    {
        double x, y; // private variables

        #pragma omp for
```

```c
printf("Running with %d threads...\n", num_threads);

#pragma omp parallel
{
    double x, y; // private variables

    #pragma omp for
    for(long i = 0; i <= n; i++)
    {
        x = i * dx;
        y = exp(x) * cos(x) * sin(x) * sqrt(5 * x + 6.0);

        // Every 1,000,000th iteration
        if(i % 1000000 == 0)
        {
            #pragma omp critical
            {
                fprintf(fp, "i=%ld x=%f y=%f\n", i, x, y);
            }
        }
    }
}

fclose(fp);
printf("Done. Output written to calculations.txt\n");

return 0;
```

```
/usr/bin/ld: T9q5.c:(.text+0x28b): undefined reference to `sqrt'
collect2: error: ld returned 1 exit status
eshaa_ntl31s1@LAPTOP-DM53QFHD:/mnt/c/Users/eshaa_ntl31s1/OneDrive/Documents$ gcc -fopenmp T9q5.c -o T9q5 -lm
eshaa_ntl31s1@LAPTOP-DM53QFHD:/mnt/c/Users/eshaa_ntl31s1/OneDrive/Documents$ ./T9q5
Usage: ./T9q5 <num_threads>
eshaa_ntl31s1@LAPTOP-DM53QFHD:/mnt/c/Users/eshaa_ntl31s1/OneDrive/Documents$ ./T9q5 4
Running with 4 threads...
Done. Output written to calculations.txt
eshaa_ntl31s1@LAPTOP-DM53QFHD:/mnt/c/Users/eshaa_ntl31s1/OneDrive/Documents$
```

```
i=0 x=0.000000 y=0.000000
i=1000000 x=0.010000 y=0.024842
i=51000000 x=0.510000 y=2.074617
i=76000000 x=0.760000 y=3.342615
i=26000000 x=0.260000 y=0.870562
i=2000000 x=0.020000 y=0.050381
i=52000000 x=0.520000 y=2.126979
i=77000000 x=0.770000 y=3.387575
i=27000000 x=0.270000 y=0.912959
i=3000000 x=0.030000 y=0.076617
i=53000000 x=0.530000 y=2.179454
i=78000000 x=0.780000 y=3.431721
i=28000000 x=0.280000 y=0.955950
i=4000000 x=0.040000 y=0.103553
i=54000000 x=0.540000 y=2.232012
i=29000000 x=0.290000 y=0.999524
i=79000000 x=0.790000 y=3.475008
i=5000000 x=0.050000 y=0.131190
i=55000000 x=0.550000 y=2.284624
i=30000000 x=0.300000 y=1.043668
i=80000000 x=0.800000 y=3.517389
i=6000000 x=0.060000 y=0.159528
i=56000000 x=0.560000 y=2.337261
i=31000000 x=0.310000 y=1.088372
i=81000000 x=0.810000 y=3.558817
i=7000000 x=0.070000 y=0.188567
i=32000000 x=0.320000 y=1.133620
i=57000000 x=0.570000 y=2.389892
i=82000000 x=0.820000 y=3.599245
i=8000000 x=0.080000 y=0.218308
i=33000000 x=0.330000 y=1.179400
```